

Module - I

Introduction

Basic Terminology

Data: Raw facts stored in memory.

Data Structure: A method to organize and store data efficiently.

Instruction: A program statement.

Instruction Types:

- Declaration Instruction
- I/O Instruction
- Arithmetic Instruction
- Control Instruction.

Data Types & Classification

A data type defines the set of values a variable can hold.

- Primitive (Built-in): int, float, char, double, void.
- Derived (Built from Primitive): Arrays, Pointers, Functions.
- User-defined (Custom): struct, union, enum, typedef.

Measuring Performance of Algorithm

Algorithm analysis is an important part of computational complexity theory, which provides theoretical estimation for the required resources of an algorithm to solve a specific computational problem. It is the determination of the amount of time and space resources required to execute it.

Q. Why analysis is required?

- Predicting behaviour of an algorithm without implementing.
- Analysis is only approximation, there're many more factors.
- It helps us in determining the best algorithm to solve a programming problem.

Type of Algorithm Analysis

- Certain input takes less time for an algorithm to accomplish its task i.e. best case. $[O]$
- Certain input takes maximum time for an algorithm to accomplish its task i.e. worst case. $[O]$
- All the other cases in which algorithm takes time for random/typical input is average case. $[O]$

Time Complexity

- It is measure of rate of change in time with respect to change in input size.
- Instead of measuring runtime in seconds, which varies based on hardware or processor speed, it counts the number of basic operations performed by the algorithm.
- It provides a standardized way to compare two different algorithms solving the same problem to see which is more efficient.
- It helps to predict how an algorithm will perform when processing massive amounts of data.
- There are various asymptotic notation to represent time complexity of an algorithm.

Asymptotic Notation

It is used to analyze performance of algorithm in term of input size. We do not calculate the actual time taken by the algorithm to solve the problem. Rather, we're interested in how the time taken by increasing with input size.

Big O Notation:

It provides upper bound on the growth rate of time and also ~~provides~~ represents the worst case.

Omega Notation:

It provides lower bound on the growth rate of time and represents the best case.

Theta Notation:

It provides both upper and lower bound on the growth rate of time and represents the average case.

(1)

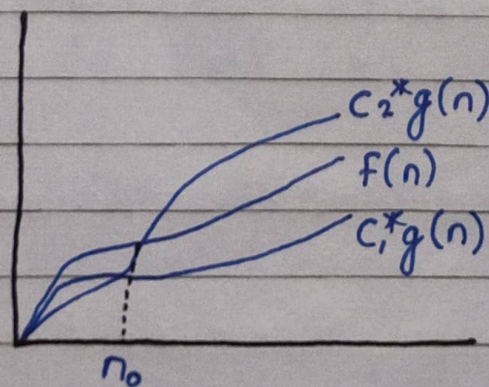


Fig.: ~~Big~~ Theta Notation

$$c_1 * g(n) \leq f(n) \leq c_2 * g(n)$$

c_1 & c_2 are the constant for all $n \geq n_0$.

$$f(n) = an^3 + bn + c$$

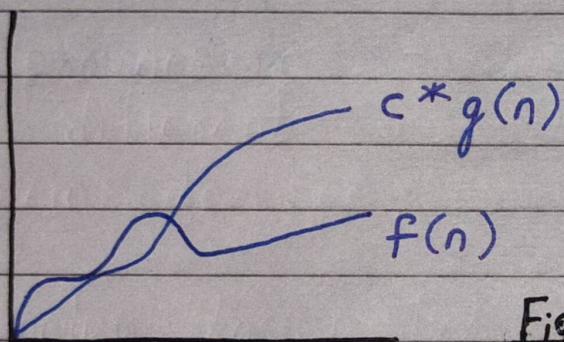
$$g(n) = 3n^3 + 2n + 5$$

$$g(n) = n^3$$

$$f(n) = \Theta(g(n^3))$$

$$\Theta(g(n^3)) = \Theta(n^3).$$

(2)



$$O(g(n)) = f(n)$$

$$0 \leq f(n) \leq g(n) \quad \forall n \geq n_2$$

Fig.: Big O Notation

$$f(n) = 2n^2 + 3n + 4$$

$$3f \quad n^2 > f(n)$$

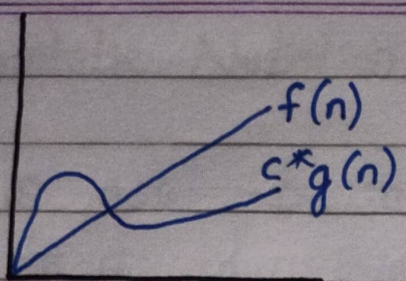
$$3n^2 > 2n^2 + 3n + 4$$

$$g(n) = 2n^2 + 3n + 4$$

$$O(g(n)) = O(n^2)$$

$$O(n^2).$$

(3)



$$\Omega(g(n)) = f(n)$$

$$c * g(n) \leq f(n) \quad \forall \quad n \geq n_0$$

Fig.: Omega Notation

$$f(n) = 2n^2 + 3n + 5$$

$$g(n) = n^2$$

$$\Omega(g(n)) = \Omega(n^2)$$

$$1 * n^2 < f(n)$$

$$1 * n^2 < 2n^2 + 3n + 5$$

$$\Omega(n^2).$$

Calculating $\text{Big } \mathcal{O}$ for Factorial

```
f = 1  $\longrightarrow$   $c_1$   
if (n == 0)  $\longrightarrow$   $c_2$   
return 1
```

$$f(n) = c_1 + c_2 + n \cdot c_3 + c_4$$
$$\Rightarrow f(n) = n \cdot c_3 + c$$
$$[c = c_1 + c_2 + c_4]$$

```
for (i = 1 to n)  
  f = i * f  $\longrightarrow$   $c_3$   
return f  $\longrightarrow$   $c_4$ 
```

$$g(n) = n$$
$$f(n) = \mathcal{O}(g(n)) = \mathcal{O}(n).$$

$\text{Big } \mathcal{O}$ of Insertion Sort Algorithm

```
for (i = 1 to n)  
  temp = A[i]  $\longrightarrow$   $c_1$ 
```

```
    for (j = i - 1 to 0)  
      if (A[j] > temp)  
        A[j+1] = A[j]  
      else  
        break
```

} $\longrightarrow i \cdot c_2$

```
  A[j+1] = temp  $\longrightarrow$   $c_3$ 
```

Cost for step $i = c_1 + i \cdot c_2 + c_3 = i \cdot c_2 + c$ [$c = c_1 + c_3$]

$$f(n) = \sum_{i=1}^n (i \cdot c_2 + c) = n^2 c_2 + n \cdot c_1$$

$$g(n) = n^2$$

$$f(n) = \mathcal{O}(g(n)) = \mathcal{O}(n^2).$$

Basic Formate Specifier

<u>Data Type</u>	<u>Formate Specifier</u>
int	→ %d or %i
float	→ %f
char	→ %c
double	→ %lf
String	→ %s

Program 1-1:

```
#include <stdio.h>
```

```
int  
void main()  
{
```

```
    char a,b,c;
```

```
    a = 65;
```

```
    b = 66;
```

```
    c = 67;
```

```
    printf("%c", a); // A
```

```
    printf("%c", b); // B
```

```
    printf("%c", c); // C
```

```
    return 0;
```

```
}
```


Program-2:

```
#include <stdio.h>
```

```
int main()  
{
```

```
    int arr[] = {10, 20, 30, 40, 50};
```

```
    int size = sizeof(arr);
```

```
    int length = sizeof(arr) / sizeof(int);
```

```
    printf("Size of array in bytes: %d", size); // 20  
    printf("\n Length of array: %d", length); // 5
```

```
    return 0;
```

```
}
```

Program-3:

```
#include <stdio.h>
```

```
int main() {
```

```
    float num = 3.5;
```

```
    printf("%f", num); // 3.5
```

```
    printf("\n %f", num); // 3.5
```

```
    printf("\n %.3f", num); // 3.500
```

```
    return 0;
```

```
}
```

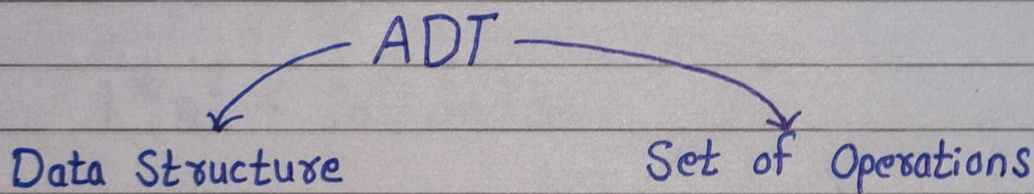

Data Structure

Data Structure is a particular way of storing and organising data in a computer so that it can be used effeciently.

Classification of Data Structure:

1. Linear Data Structure (Array, Linked List, Stack & Queue).
2. Non-linear Data Structure (Tree & Graph).

Abstract Data Type (ADT)



- All primitive data types support basic operation- like addition, subtraction, etc.
- The system is providing the implementation for the primitive data type.
- For non-primitive data type we also need to define operation.
- Combination of data structure and their operation are known as ADT.

Array

An array is a collection of similar element. It is a linear collection of item.

Array ADT:

An array ADT is defined for both Data Structure and set of operation.

The set of operations:

- Creation of array of specific size.
- Retrieve a particular item from given index.
- Store an item at given index.
- Modify item of given index.
- Count items stored in the array.

Operation of on Linear Array

1. Traversal
2. Searching
3. Insertion
4. Deletion
5. Sorting
6. Merging

1. Traversing Linear Array

If we want to print element of array or count number of element in the array, then this can be achieved by traversing array.

Algorithm:

LA $\rightarrow$ Linear Array

LB $\rightarrow$ Lower Bound (Starting Index)

UB $\rightarrow$ Upper Bound (Ending Index)

Step-1: Start.

Step-2: Set $K = LB$.

Step-3: Repeat step 4 & 5 while $K \leq UB$.

Step-4: Apply process to $LA[K]$.

Step-5: Set $K = K + 1$ (or $K += 1$).

Step-6: Stop.

Program:

```
#include <stdio.h>
```

```
int main ()  
{
```

```
    int LA[] = {10, 20, 30, 40, 50};
```

```
    int LB = 0, UB = 4;
```

```
    int k = LB;
```

```
    while (k <= UB)  
    {
```

```
        printf("Element at %d = %d \n", k, LA[k]);  
        k += 1;
```

```
    }
```

```
    return 0;
```

```
}
```

Output:

Element at 0 = 10

Element at 1 = 20

Element at 2 = 30

Element at 3 = 40

Element at 4 = 50

2. Insertion in Linear Array

Insertion refers to the operation of adding another element to linear array. It can be done on three ways;

- i. Insert an element at beginning of array.
- ii. Insert an element at the end of array.
- iii. Insert an element at given location (i.e. index).

(i) Inserting at beginning of array [Worst Case]

To insert an element at index 0 i.e. at beginning, all existing elements must shift one position to the right.

Algorithm:

Step-1: Start

Step-2: Check if array is full or not.

Step-3: Set $i = n - 1$.

Step-4: Repeat step 5 & 6 while $i \geq 0$.

Step-5: Set $LA[i+1] = LA[i]$.

Step-6: Set $i = i - 1$ (or $i -= 1$).

Step-7: Set $LA[0] = \text{New_element}$.

Step-8: Set $n = n + 1$.

Step-9: Stop.

Program:

```
#include <stdio.h>
```

```
int main()  
{
```

```
    int LA[10] = {10, 20, 30, 40, 50};
```

```
    int n = 5, maxSize = 10, new_element = 5;
```

```
    if (n >= maxSize)  
    {
```

```
        printf("Array is full.");  
        return 0;
```

```
    }
```

```
    for (int i = n - 1; i >= 0; i--)  
    {
```

```
        LA[i + 1] = LA[i];
```

```
    }
```

```
    LA[0] = new_element;
```

```
    n += 1;
```

```
    return 0;
```

```
}
```


(ii) Inserting at the End of Array [Best Case]

Inserting at the end takes $O(1)$ time because no elements need to be shifted.

Algorithm:

Step-1: Start

Step-2: Check if the array is full or not.

Step-3: Set $LA[n] = \text{new_element}$.

Step-4: Set $n = n + 1$.

Step-5: Stop.

Program:

```
#include <stdio.h>
```

```
int main()  
{
```

```
    int LA[10] = {10, 20, 30, 40, 50};
```

```
    int n = 5, maxSize = 10, new_element = 60;
```

```
    if (n >= maxSize)  
    {
```

```
        printf("Array is full.");  
        return 0;
```

```
}
```



```
LA[n] = new_element;
```

```
n += 1;
```

```
for (int i = 0; i < n; i++)  
{
```

```
    printf("Element at %d = %d \n", i,  
          LA[i]);
```

```
}
```

```
return 0;
```

```
}
```

Output:

Element at 0 = 10

Element at 1 = 20

Element at 2 = 30

Element at 3 = 40

Element at 4 = 50

Element at 5 = 60

(iii) Inserting at a Given Location / Index [Avg. Case]

To insert an element at a given index i.e. position (say pos), elements from index $n-1$ down to pos must be shifted one position to the right.

Algorithm:

Step-1: Start

Step-2: Check if the array is full or not.

Step-3: Set $i = n - 1$.

Step-4: Repeat steps 5 & 6 while $i \geq \text{pos}$.

Step-5: Set $LA[i+1] = LA[i]$.

Step-6: Set $i = i - 1$.

Step-7: Set $LA[\text{pos}] = \text{new_element}$.

Step-8: Set $n = n + 1$.

Step-9: Stop.

Program:

```
#include <stdio.h>
```

```
int main()  
{
```

```
    int LA[10] = {10, 20, 30, 40, 50};
```

```
    int n = 5, maxSize = 10;
```

```
    int pos = 2, new_element = 25;
```

```
    if (n >= maxSize)  
    {
```

```
        printf("Array is full.");  
        return 0;
```

```
    }
```

```
    for (int i = n - 1; i >= pos; i--)  
    {
```

```
        LA[i + 1] = LA[i];
```

```
    }
```

```
    LA[pos] = new_element;  
    n += 1;
```

```
    for (int i = 0; i < n; i++)  
    {
```

```
        printf("Element at %d = %d\n", i,  
              LA[i]);
```

```
    }
```

```
    return 0;
```

```
}
```


3. Deletion in Linear Array

Deletion refers to removing an existing element from an array and shifting elements left to fill the gap.

Algorithm (Deleting at position pos):

Step-1: Start.

Step-2: Set $item = LA[pos]$.

Step-3: Repeat steps 4 & 5 for $i = pos$ to $n-2$.

Step-4: Set $LA[i] = LA[i+1]$.

Step-5: Set $i = i+1$.

Step-6: Set $n = n-1$.

Step-7: Stop.

Program:

```
#include <stdio.h>
```

```
int main()  
{
```

```
    int LA[] = {10, 20, 30, 40, 50};  
    int n = 5, pos = 2;
```

```
    for (int i = pos; i <= n - 2; i++)  
    {  
        LA[i] = LA[i+1];  
    }
```

```
    n -= 1;
```

```
    for (int i = 0; i < n; i++)  
    {  
        printf("Element at %d = %d \n", i,  
               LA[i]);  
    }
```

```
    return 0;  
}
```

Output:

```
Element at 0 = 10  
Element at 1 = 20  
Element at 2 = 40  
Element at 3 = 50
```


4. Searching in Linear Array

Searching refers to finding the location/index of a given element (item) in an array. There are two primary types of searching techniques in arrays:

i. Linear (Sequential) Search

ii. Binary Search

i. Linear Search

In this, we check each element one by one from start to end. It works on both sorted and unsorted arrays.

Algorithm:

Step-1: Start

Step-2: Set $pos = -1$

Step-3: Repeat step 4 for $i = 0$ to $n-1$.

Step-4: Check if $LA[i] == item$, then set $pos = i$ and exit loop.

Step-5: Check if $pos \neq -1$ then print element found at index pos else element not found.

Step-6: Stop

LA $\rightarrow$ Linear array

n $\rightarrow$ Number of elements in array.

item $\rightarrow$ Element to search.

pos $\rightarrow$ Position/Index of item.

Program:

```
#include <stdio.h>
```

```
int main() {
```

```
    int LA[] = {10, 20, 30, 40, 50};
```

```
    int n = 5, item = 30, pos = -1;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (LA[i] == item) {
```

```
            pos = i;
```

```
            break;
```

```
        }
```

```
    }
```

```
    if (pos != -1) {
```

```
        printf("Element found at %d", pos);
```

```
    }
```

```
    else {
```

```
        printf("Element not found");
```

```
    }
```

```
    return 0;
```

```
}
```


ii. Binary Search

In this we use divide and conquer approach. It works only on sorted arrays by repeatedly dividing the search space in half.

Algorithm:

Step-1: Start

Step-2: Set $low = 0$, $high = n - 1$, $pos = -1$.

Step-3: Repeat steps 4 to 6 while $low \leq high$.

Step-4: Set $mid = (low + high) / 2$

Step-5: Check if $LA[mid] == item$, true then set $pos = mid$ and exit loop.

Step-6: If $LA[mid] < item$, set $low = mid + 1$
else set $high = mid - 1$.

Step-7: If $pos \neq -1$ then print element found
else print element not found.

Step-8: Stop.

$low \rightarrow$ Lower bound (starting index)

$high \rightarrow$ Upper bound (ending index)

$mid \rightarrow$ Middle index.

Program:

```
#include <stdio.h>
```

```
int main()  
{
```

```
    int LA[] = {10, 20, 30, 40, 50};
```

```
    int n = 5, item = 30, low = 0, high = n - 1, mid,  
        pos = -1;
```

```
    while (low <= high)  
    {
```

```
        mid = (low + high) / 2;
```

```
        if (LA[mid] == item)  
        {
```

```
            pos = mid;  
            break;
```

```
        }
```

```
        else if (LA[mid] < item)
```

```
            low = mid + 1;
```

```
        else
```

```
            high = mid - 1;
```

```
    }
```

```
    if (pos != -1)
```

```
        printf("Element found at %d", pos);
```

```
    else
```

```
        printf("Element not found.");
```

```
    return 0;
```

```
}
```


5. Sorting Linear Array

Sorting refers to arranging the elements of an array in a specific order either ascending or descending.

Algorithm (Bubble Sort):

Step-1: Start

Step-2: Repeat steps 3 & 4 for $i=0$ to $n-2$.

Step-3: Repeat step 4 for $j=0$ to $n-i-2$.

Step-4: If $LA[j] > LA[j+1]$, then $temp = LA[j]$,
 $LA[j] = LA[j+1]$, $LA[j+1] = temp$.

Step-5: Stop.

$LA \rightarrow$ Linear array

$n \rightarrow$ Number of elements in the array.

i & $j \rightarrow$ Loop counters

$temp \rightarrow$ Temporary variable used for swapping element.

Program (Bubble Sort):

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int LA[] = {50, 20, 40, 10, 30};
```

```
int n = 5, i, j, temp;
```

```
for (i = 0; i < n - 1; i++)
```

```
{
```

```
for (j = 0; j < n - i - 1; j++)
```

```
{
```

```
if (LA[j] > LA[j + 1])
```

```
{
```

```
temp = LA[j];
```

```
LA[j] = LA[j + 1];
```

```
LA[j + 1] = temp;
```

```
}
```

```
}
```

```
}
```

```
printf("Sorted Array: ");
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
printf("%d ", LA[i]);
```

```
}
```

```
return 0;
```

```
}
```